

Introspecție. Programare concurentă

Conf. Univ. Arthur Molnar

Universitatea Babeș-Bolyai

2025

Privire de ansamblu

- 1 Introspecția în Java (eng. Java Reflection)
 - Introspecție și reflecție (eng. introspection and reflection)
 - API-ul Java Reflection
- 2 Concurență și paralelism
 - Concurență și paralelism
 - Procese
 - Fire de execuție (eng. *threads*)
 - Programarea pe mai multe fire de execuție în Java

Încărcarea în memorie a claselor Java I

- Un program compilat în limbajul Java constă dintr-o mulțime de fișiere având extensia **.class**. Fiecare fișier **.class** corespunde unei clase a programului.
- Nu toate clasele sunt încărcate în memorie la momentul lansării în execuție a programului. De regulă, acest lucru se întâmplă în momentul primei referențieri a clasei (încărcare "leneșă", eng. **lazy loading**)
- Această tehnică scade necesarul de memorie al programului și reduce timpul necesar lansării în execuție.
- Încărcarea claselor în memorie este efectuată de componenta **class loader**; aceasta este inițializată de mașina virtuală Java și se ocupă de încărcarea în mod dinamic a claselor.
- Când mașina virtuală are nevoie de definiția unei clase (câmpuri, metode, instrucțiuni) este responsabilitatea componentei de încărcare a claselor (eng. **class loader**) să identifice locația clasei folosind numele său complet și să îi încarce definiția în mașina virtuală.

Încărcarea în memorie a claselor Java II

- Există trei tipuri de componente pentru încărcarea claselor (eng. **class loaders**):
 - **Bootstrap/Primordial ClassLoader**: inițializează procesul de încărcare al claselor și încarcă clasele de sistem din fișierul în care acestea sunt stocate (de regulă **rt.jar**).
 - **Extension ClassLoader**: încarcă extensiile claselor din nucleul Java (eng. *Java core classes*) din biblioteca de extensii.
 - **Application/System ClassLoader**: încarcă clasele aplicațiilor client Java.
- Componenta de încărcare a claselor **Bootstrap/Primordial ClassLoader** este scrisă în C/C++, în timp ce celelalte componente pentru încărcarea claselor sunt scrise în Java.

Încărcarea în memorie a claselor Java III

- Fiecare componentă pentru încărcarea claselor (eng. **class loader**) are o componentă de încărcare a claselor părinte.
- Platforma Java utilizează un model de delegare pentru încărcarea claselor: atunci când se încarcă o clasă, componenta responsabilă delegă cererea către componenta părinte în mod recursiv.
- Dacă nici unul din componentele părinte nu localizează clasa ce trebuie încărcate, atunci componenta-copil (eng. **child class loader**) va căuta definiția clasei în cadrul sistemului de fișiere.
- În cazul în care nici unul din componentele existente nu găsesc definiția clasei, se va arunca o excepție de tip **java.lang.ClassNotFoundException**.

Încărcarea în memorie a claselor Java IV

- Acest comportament permite flexibilizarea modului în care sunt încărcate clasele (ex. nu toate clasele sunt încărcate din fișiere **.class**)

Exemple pentru componentele de încărcare a claselor Java

- <https://www.oodlestechnologies.com/blogs/class-loader-delegation-in-java-with-example/>
- În unele cazuri, byte-code-ul Java este generat pe loc folosind unelte precum **cglib**

Ciclul de viață al obiectelor Java I

- 1 Clasa este localizată pe disc (sau rețea, sau este generată pe loc) și definiția ei este încărcată în memorie
- 2 Sunt inițializate câmpurile statice și rulate blocurile de cod marcate ca fiind **static**. Acestea aparțin de clasa în sine și nu de un anumit obiect (deci nu este nevoie de un constructor pentru a le construi, și nu au acces la o referință **this**).
- 3 Când primul obiect al unei clase este instanțiat sau când o metodă/câmp marcată **static** este accesată pentru prima dată, definiția clasei este încărcată în memorie.
- 4 Când un obiect al clasei este creat folosind cuvântul cheie **new**, se alocă memoria necesară în zona heap împreună cu o referință care permite mașinii virtuale să păstreze accesul la obiect.

Ciclul de viață al obiectelor Java II

- 5 Obiectul este disponibil în cadrul programului și este utilizat atâta timp cât este necesar; mai precis, cât timp există cel puțin o referință validă în cadrul programului către acel obiect.
- 6 Odată ce obiectul nu mai poate fi referit din programul Java (nu mai există referințe către el, sau toate referințele către obiect au ieșit din domeniul de vizibilitate), componenta de **garbage collection** este liberă să o dealoce.
- 7 În momentul în care nu mai există referințe valide către o clasă, atunci și obiectul corespunzător clase în sine este poate fi dealocat de către componenta de **garbage collection**.

Introspecție și reflecție

- Această secțiune se bazează pe documentația disponibilă la adresa [the Reflection API](#).
- "Mecanismul de reflecție este utilizat de programele care necesită abilitatea de a examina sau modifica comportamentul în execuție al aplicațiilor ce rulează în mașina virtuală Java." ([documentația Java](#))
- **Reflecția** este abilitatea unui program de a examina și modifica structura și comportamentul (proprietățile interne – câmpuri și metode) ale obiectelor în timpul execuției programului.
- Mecanismul de introspecție permite inspectarea tipurilor sau metadatelor asociate pe durata execuției programului. Mecanismul de reflecție face un pas adițional și permite modificarea acestora.

Avantajele utilizării mecanismului de reflecție I

- Permite crearea de instanțe (obiecte) ale unor clase externe aplicației pe baza numelor acestora → *extensibilitate*.
- Mediile de dezvoltare vizuale (e.g., **SceneBuilder** utilizează informațiile despre membrii claselor pentru a ajuta dezvoltatorii.
- Depanatoarele pot accesa membrii privați ai unei clase pentru a putea examina/modifica valorile acestora în cadrul sesiunii de depanare.
- Cadrele de aplicație pentru testare (eng. **testing frameworks**) pot apela și utiliza un set de API-uri definite pentru a strânge informații despre acoperirea testelor (eng. **test coverage**).

Avantajele utilizării mecanismului de reflecție II

- Cadrul de aplicație pentru testare JUnit folosește reflecția pentru a descoperi metodele anotate ca fiind metode de testare (folosind adnotarea **@Test** și a le apela în cadrul sesiunii de testare.
- Mecanismul de reflecție poate fi utilizat pentru a mapa numele coloanelor dintr-un ResultSet returnat de o interogare SQL efectuată prin JDBC cu câmpurile unui obiect corespunzător Java.
- JavaFX utilizează mecanismul de reflecție pentru a crea obiectele corespunzătoare componentelor grafice declarate în fișierele FXML.

Dezavantajele utilizării mecanismului de reflecție

- Operațiile care se bazează pe mecanismul de reflecție sunt executate mai lent decât cele native deoarece mașina virtuală nu poate efectua întreaga gamă de optimizări pe cod ce utilizează reflecția.
- Mecanismul de reflecție nu poate fi utilizat în medii cu securitate ridicată, deoarece necesită acordarea anumitor permisiuni pentru programul ce încorporează acest mecanism.
- Utilizarea reflecției încalcă principiile de abstractizare al codului prin oferirea posibilității de executare al codului care altfel nu ar trece de compilare (ex. accesarea câmpurilor și metodelor **private**); aceasta poate duce la probleme de mentenanță al codului, deoarece atributele marcate **private** sunt considerate ca fiind detalii de implementare ce pot fi modificate în orice moment.

Concluzie

Când să folosim mecanismul de reflecție

Dacă efectuarea unei operații sau implementarea unui comportament este posibilă fără utilizarea mecanismului de reflecție, atunci această implementare este dezirabilă.

Ce se poate face folosind reflecția în Java?

Putem

- Examina clasa unui obiect în timpul rulării programului.
- Instanția un obiect al unei clase cunoscând numele clasei.
- Examina metodele și atributele unui obiect în timpul execuției programului – inclusiv a celor marcate ca fiind **private**
- Invoca o metodă al cărui nume îl cunoaștem pentru un obiect dat.
- Modifica clasa de protecție al constructorilor, metodelor și atributelor unei clasei (ex. un atribut **private** poate fi marcat ca fiind **public** și apoi valoarea sa poate fi modificată)

Obiectul `java.lang.Class` I

- Pentru fiecare tip de obiect mașina virtuală Java instanțiază un obiect al clasei `java.lang.Class`
- Această instanță poate fi utilizată pentru:
 - A inspecta membrii clasei la executarea programului
 - A crea noi instanțe (obiecte) ale clasei
- De asemenea, această clasă reprezintă și punctul de intrare pentru utilizarea API-ului de reflecție în Java.
- O clasă într-un program Java este practic un obiect al meta-clasei `java.lang.Class`

```
Class studentClass = Student.class;  
Student s = new Student(2, "Pop Andrei", 9.2);  
Class studentClass2 = s.getClass();
```

Obiectul `java.lang.Class` II

- Instanțe ale clasei `java.lang.Class` pot fi obținute și prin utilizarea metodei statice

`Class.forName(String numeClasa)`, atunci când cunoaștem numele complet al clasei dorite.

```
try {  
    Class studentClass2 =  
        Class.forName("reflection.Student");  
} catch (ClassNotFoundException e) {  
    e.printStackTrace();  
}
```


Obiectul java.lang.Class III

Class
...
+getName() : String
+getFields() : Field[*]
+getMethod() : Method[*]
+getConstructors() : Constructor[*]
<u>+forName(className : String) : Class</u>
...

Obținerea informațiilor relevante pentru pachete

- Se utilizează metoda `Package getPackage()`.
- Obiectul de tip `java.lang.Package` include informații precum:
 - Numele pachetului
 - Informații privind versiunea implementării și a specificației pachetului
 - Anotările existente

```
Package p = studentClass.getPackage();
```

Accesarea clasei de bază sau a interfețelor implementate

- Pentru a obține obiectul clasă corespunzător clasei de bază putem utiliza metoda `getSuperclass()`.
- Aceasta returnează obiectul `java.lang.Class` corespunzător tipului clasei de bază – toate mecanismele de reflecție pot fi utilizate (și) pe acest tip.
- Lista interfețelor implementate de o clasă poate fi obținută prin apelarea metodei `Class[] getInterfaces()`, care returnează doar acele interfețe care au fost declarate în mod explicit de clasa pe care este apelată metoda.
- Lista completă a interfețelor implementate, care include interfețele implementate prin clasele de bază se poate obține apelând metoda în mod recursiv pe ierarhia de moștenire.

```
Class superClass = studentClass.getSuperclass();  
Class [] interfaces = studentClass2.getInterfaces();
```

Apelarea constructorilor I

- Accesarea listei constructorilor se face prin apelarea metodei `Constructor[] getConstructors()`.
- Un obiect de tipul `java.lang.reflect.Constructor` modelează și permite accesul la un singur constructor al unei clase.
- Putem alege constructorul dorit folosind parametrii pe care acesta îi acceptă.

```
Constructor[] constructors = studentClass.getConstructors();
```

```
try {  
    Constructor constructor =  
        studentClass.getConstructor(String.class);  
} catch (NoSuchMethodException e) {  
    e.printStackTrace();  
}
```

Apelarea constructorilor II

Parametrii constructorilor

- Se utilizează metoda `Class[] getParameterTypes()`

```
Class [] parameterTypes = constructor.getParameterTypes();
```

Instanțierea obiectelor folosind un obiect de tipul `java.lang.reflect.Constructor`

- Se apelează metoda `newInstance()`, care acceptă un număr variabil de parametri.
- Numărul și tipul parametrilor furnizați trebuie să fie exact aceeași ca cei ai constructorului apelat.

```
Constructor constructor = studentClass.getConstructor(  
    int.class, String.class, double.class);  
Student newObject =  
    (Student) constructor.newInstance(2, "Marin Ioana", 9.4);
```

Utilizarea obiectelor `java.lang.reflect.Field` I

- Se utilizează metoda `Field[] getFields()` pentru a obține un vector al tuturor câmpurilor cu modificatorul `public` din cadrul clasei. Pentru a accesa câmpurile marcate `private` este necesară apelarea metodei `getDeclaredFields()`.
- Dacă numele câmpului dorit este cunoscut, acesta poate fi accesat direct folosind metoda `getField(<field_name>)` sau metoda `getDeclaredField(<field_name>)`;
- Aceste metode returnează doar câmpurile declarate în cadrul clasei, nu și cele moștenite din clasele de bază.
- Numele și tipurile câmpurilor marcate `public` pot fi obținute folosind metodele `getName()` și `getType()`.

Utilizarea obiectelor `java.lang.reflect.Field` II

- Valorile câmpurilor marcate `public` pot fi accesate sau setate pentru o instanță a clasei (obiect). Același lucru se poate face și pentru câmpurile marcate `private`, doar că acestea trebuie întâi să fie marcate ca fiind accesibile prin utilizarea metodei `setAccessible(boolean)`.

Utilizarea obiectelor java.lang.reflect.Field III

```
// private fields
Field[] fields = studentClass.getDeclaredFields();
try {
    // get field with a given name
    Field f = studentClass.getDeclaredField("average");
    // make the private field accessible
    f.setAccessible(true);
    Student st = new Student(2, "Marin Ioana", 9.4);
    // get value for fields
    Object value = f.get(st);
    // set value for field
    f.set(st, 10);
    // now st.getAverage() will return 10
} catch (NoSuchFieldException e) { /*...*/ }
catch (IllegalAccessException e) { /*...*/ }
```


Utilizarea obiectelor `java.lang.reflect.Method` I

- Se utilizează metoda `Method[] getMethods()` pentru a returna toate metodele marcate `public` al clasei și metoda `Method[] getDeclaredMethods()` pentru a returna inclusiv metodele marcate `private`.
- La fel ca și în cazul câmpurilor, putem obține o referință spre o metodă publică sau privată folosind numele și lista parametrilor acceptați în cadrul metodei `getMethod(String name, Class[] parameterTypes)` sau `getDeclaredMethod(String name, Class[] parameterTypes)`, respectiv.
- Tipurile parametrilor se pot obține folosind metoda `getParameterTypes()` în timp ce tipul de dată returnat se obține apelând metoda `getReturnType()`.
- Metoda în sine poate fi apelată folosind metoda `invoke()`. Aceasta se apelează folosind obiectul pe care este invocată metoda împreună cu lista parametrilor actuali.

Utilizarea obiectelor `java.lang.reflect.Method` II

```
Method[] methods = studentClass.getMethods();

try {
    // getting a method by name and parameters
    Method m = studentClass.getMethod("compareTo",
                                       Object.class);

    // getting the parameter types
    Class[] parameterTypes = m.getParameterTypes();

    // getting the return type
    Class returnType = m.getReturnType();
}
```

Utilizarea obiectelor java.lang.reflect.Method III

```
// invoking the method
Student st1 = new Student(2, "Marin Ioana", 8.4);
Student st2 = new Student(3, "Bojan Paul", 9.1);
System.out.println(m.invoke(st1, st2));
} catch (NoSuchMethodException e) { /*...*/ }
catch (IllegalAccessException e) { /*...*/ }
catch (InvocationTargetException e) { /*...*/ }
```

Utilizarea obiectelor `java.lang.annotation.Annotation`

- Adnotările unei clase pot fi obținute prin apelarea metodei `Annotation[] getAnnotations()` pe un obiect de tipul `java.lang.Class`.
- Adnotările metodelor pot fi obținute prin apelarea metodei `Annotation[] getDeclaredAnnotations()` pe un obiect de tipul `java.lang.reflect.Method`, iar cele pentru câmpurile clasei prin apelarea metodei `Annotation[] getDeclaredAnnotations()` apelate pentru un obiect de tipul `java.lang.reflect.Field`.

Exemplu

Utilizarea mecanismului de reflecție pentru citirea/scrierea generică în fișiere CSV (comma-separated values)

`lecture.examples.lecture11.Main.java`

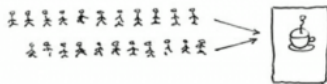
Concurență și paralelism I

- **Paralelismul** se referă la utilizarea mai multor procesoare/nuclee de procesor (eng. CPU cores) pentru a executa calcule mai repede.
- Un sistem paralelizat poate executa mai multe operații *simultan*.
- **Concurența** se referă la gestionarea accesului la starea comună din mai multe fire de execuție (eng. threads) ale aceluiași proces.
- Într-un mediu concurrent operațiile pot fi executate simultan, dar, ținând cont de faptul că unele resurse sunt partajate, acest lucru nu e tot timpul posibil.
- Un calculator modern are în fiecare moment multe procese și fire de execuție active (acest lucru poate fi observat în *Task Manager* în Windows sau folosind *Activity Monitor* în macOS).

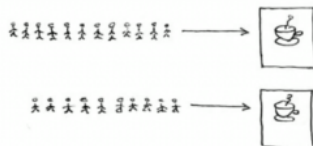
Concurență și paralelism II

CONCURRENCY | PARALLELISM

Concurrent = Two Queues One Coffee Machine



Parallel = Two Queues Two Coffee Machines



© Jan Armstrong 2018

Figure: Sursa figurii: [Flavors of Concurrency in Java by Oleg Shelajev](#)

Procese I

- Un **proces** este o instanță a unui program aflat în execuție.
- Fiecare proces are propriul mediu de execuție precum și propriul set de resurse private (ex: spațiu de memorie, handle-uri de fișiere).
- Procesele sunt entități independente, izolate care sunt gestionate de sistemul de operare.
- Procesele pot comunica între ele folosind resurse precum *sockets*, *pipe-uri* cu și fără nume, *memorie partajată* (eng. *shared memory*) etc. Aceste mecanisme se numesc *Inter Process Communication (IPC)*.

Procese II

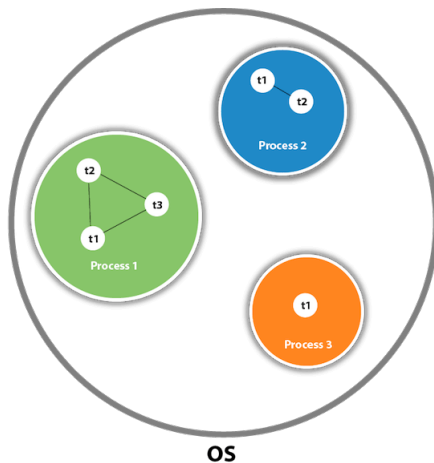


Figure: Sursa figurii: [Programare multi-thread în Java](#)

Fire de execuție (eng. *threads*) I

- O aplicație poate executa mai multe sarcini în același timp.
- Această abordare duce la o mai bună utilizare a resurselor de calcul existente.
- Calcule complexe, care durează mai mult timp nu au voie să blocheze procesul principal al programului (ex: interfața grafică nu are voie să se blocheze deoarece programul are de efectuat niște calcule care durează mai mult timp)
- **Firele de execuție** sunt asociate cu procesele și permit executarea de cod în paralel, în cadrul unui proces părinte.

Fire de execuție (eng. *threads*) II

- În cazul unui sistem care are un singur procesor cu un singur nucleu, sistemul de operare comută firul de execuție activ la un interval foarte scurt (zeci de milisecunde), care dă utilizatorului impresia că mai multe programe rulează simultan. În cadrul sistemelor multi-procesor, sau în care procesorul are mai multe nuclee (marea majoritate în ziua de azi) sistemul poate executa în paralel atâtea fire de execuție câte procesoare, sau câte nuclee au toate procesoarele sistemului de calcul.

Fire de execuție (eng. *threads*) III

- Firele de execuție își partajează resursele (memoria, fișierele deschise etc.) cu toate celelalte fire de execuție din cadrul acelui proces.
- Crearea unui fir de execuție este mai puțin complexă față de crearea unui proces.
- Fiecare proces are asociat cel puțin un fir de execuție.
- Firele de execuție asociate acelui proces au acces la memoria heap asociată procesului.
- Deoarece mașina virtuală Java alocă toate obiectele în zona heap, toate firele de execuție au acces la toate obiectele.
- Dacă un obiect este modificat de un fir de execuție, această modificare este vizibilă tuturor firelor de execuție.

Fire de execuție (eng. *threads*) IV

- Fiecare fir de execuție are propria sa stivă de execuție, care nu poate fi accesată de celelalte fire de execuție.
- Stiva păstrează doar variabilele locale primitive (care nu sunt referințe la obiecte) și referințe către obiectele stocate în zona heap.
- Memoria stivei se păstrează de regulă doar pe scurtă durată, în timp ce zona heap există din momentul lansării programului în execuție până la terminarea execuției.
- Odată ce toate metodele și-au terminat execuția, stiva de execuție este vidă.

Avantajele programării folosind mai multe fire de execuție

- Performanțe îmbunătățite - utilizare mai bună a resurselor de calcul existente, timp mai mic de execuție al programului.
- Programele răspund mai repede la acțiunile utilizatorului (ex: interfața grafică cu utilizatorul nu este niciodată blocată).
- Programele care pot utiliza mai multe fire de execuție în paralel sunt scalabile, și vor avea performanțe mai bune în sistemele multi-procesor sau cele cu procesoare cu multe nuclee.

Dezavantajele programării folosind mai multe fire de execuție

- Complexitatea programelor ce utilizează mai multe fire de execuție este mai mare; utilizarea firelor de execuție poate duce la apariția unor defecte software greu de reprodus și diagnosticat, care însă pot avea un impact mare asupra corectitudinii execuției programului (ex: calcule incorect executate din cauza modului în care execuția firelor este programată de către sistemul de operare, aspect care se poate modifica la fiecare execuție).
- Crearea și distrugerea firelor de execuție necesită resurse de calcul.
- Partajarea resurselor între firele de execuție trebuie gestionată corect, acest lucru mărin­d complexitatea programului.
- Depanarea programelor ce utilizează mai multe fire de execuție poate fi foarte complicată. De asemenea, poate fi complicată reproducerea unor defecte software raportate de testeri/utilizatori finali.

Clasa `java.lang.Object` - metode pentru programarea pe fire de execuție

- Clasa `java.lang.Object` oferă mai multe metode utilizate în programarea pe mai multe fire de execuție:
 - `wait()` - oprește execuția firului de execuție curent până la primirea unei notificări din partea unui alt fir de execuție (folosind una din metodele `notify()` sau `notifyAll()`).
 - `notify()` - notifică un alt obiect. Această metodă trezește un singur fir de execuție care așteaptă deblocarea monitorului (eng. *monitor*) acestui obiect.
 - `notifyAll()` - la fel ca metoda de mai sus, însă sunt notificate toate firele de execuție în așteptare.
- Ce este un monitor ?

Ce este un monitor?

- Un monitor este un mecanism de sincronizare care permite accesul concurent la o resursă; dezvoltate în anii 1970, reprezintă un mecanism general de sincronizare și nu sunt specifice platformei Java.
- Un monitor furnizează următoarele capacități:
 - ① *excluziune mutuală*: un singur fir de execuție are acces exclusiv la resursă.
 - ② *cooperare*:
 - firele de execuție pot aștepta (sunt blocate) până la îndeplinirea unor anumite condiții.
 - firele de execuție se pot notifica unele pe altele în momentul îndeplinirii condițiilor cerute.
- Platforma Java furnizează suport implicit pentru utilizarea mecanismului de sincronizare bazat pe monitoare.

Clasa `java.lang.Thread` I

```
public class Thread extends Object implements Runnable
```

- Interfața `java.lang.Runnable` trebuie implementată de fiecare clasă a cărei instanțe se doresc a fi executate pe un fir de execuție altul față de cel implicit.
- Metodele principale ale clasei `java.lang.Thread` sunt:
 - `run()` - metoda care conține codul care va fi executat în noul fir de execuție.
 - `start()` - pornește noul fir de execuție. În momentul în care noul fir de execuție este pornit, execuția programului este controlată de cel puțin două fire de execuție: cel implicit, și cel nou lansat.
 - `sleep()` - suspendă executarea firului de execuție pentru un număr de milisecunde dat ca parametru.

Clasa `java.lang.Thread` II

- `join()` - așteaptă ca firul de execuție să își încheie execuția.
- `yield()` - furnizează o sugestie sistemului de operare că firul de execuție curent își poate suspenda execuția pentru un anumit timp, pentru a permite executarea altor fire de execuție.
- `isAlive()` - verifică dacă firul de execuție este "viu".
- `setDaemon(boolean)` - marchează firul de execuție ca fiind un fir de execuție `daemon`. Un astfel de fir de execuție are prioritate scăzută și furnizează de regulă servicii altor fire de execuție. Dacă toate firele de execuție ale unui program sunt de acest tip, mașina virtuală Java oprește programul și își încheie execuția. Firul de execuție ce rulează mecanismul de *garbage collection* este un exemplu de astfel de fir.
- `getPriority()`, `setPriority(int)` - accesează sau modifică prioritatea firului de execuție.

Crearea noilor fire de execuție în Java

- Fiecare program Java aflat în execuție include cel puțin firul de execuție implicit; acesta este creat în mod automat de mașina virtuală Java în momentul lansării în execuție a programului.
- Acest fir de execuție apelează metoda `main()`.
- Cum putem crea fire de execuție în mod explicit:
 - Implementăm interfața `java.lang.Runnable`, iar obiectul creat îl transmitem ca parametru la crearea unui obiect de tipul `Thread`.
 - Extindem clasa `Thread` în mod direct, și implemenăm codul de executat pe noul fir de execuție în cadrul metodei `run()`.

Exemplu fire de execuție

`lecture.examples.lecture12.Main.java`